

5. Detailed Design and Implementation

5.1 - Independent FOCUS Data Generator

The generator is a separate project that produces one synthetic FOCUS-based source history for the local POC and the cloud platform. This is a logical account of the final system, not a claim that the generator was extracted before the POC was built. Its configuration identifies the anonymized reference data, output and ledger locations, date range, row and cost targets, monthly weights, and identifier salt. Publication to GCS is a separate step; the generator does not write DuckDB or Unity Catalog tables.

5.1.1 Daily data generation

The command-line interface generates the configured history, an explicit date range, the next number of days, or the remainder of the next pending month. For each date, a date-specific deterministic seed selects reference rows. The module assigns charge and billing periods, transforms supported identifiers consistently, scales cost fields, and casts the output to the source Arrow schema. It writes compressed Parquet under `datasets/focus/daily/YYYY/MM/YYYY-MM-DD.parquet`.

The file is written provisionally, hashed with SHA-256, and moved to its published local path. A ledger records its path, row count, target and actual `BilledCost`, and hash. A fingerprint of relevant settings and source hashes detects changes to the generation context. Existing output requires explicit resume behavior; resume verifies recorded files instead of silently regenerating them. A local file lock prevents concurrent generator processes from publishing to the same ledger. These controls support local reproducibility, but do not make a subsequent GCS copy a distributed transaction.

5.1.2 Monthly billing simulation

A monthly bill can be built only after every Daily file for the target month has been found and checked against its ledger entry, schema, currency, provider, date boundaries, and billed-cost total. In the `no_change` scenario, the detailed Daily rows are concatenated without aggregation. Four other scenarios introduce late usage, cost correction, a combination of both, or an unexplained difference. For each intervention, the manifest records the source and resulting billing row, input hashes, scenario, expected cost difference, and output hash. This manifest serves as an experimental oracle for reconciliation; the cloud pipeline does not read it to decide whether a difference is valid.

Monthly output follows `datasets/focus/monthly/billing-YYYY-MM.parquet`, matching the relative path used in GCS. Overwrite is explicit and copies the previous Parquet and manifest to a dated local archive. The replacement file is read back against the intended Arrow table before publication. This local version archive is distinct from the cloud platform's

currently disabled automatic RAW archival. The generator neither closes a cloud billing month nor deletes its source files.

5.1.3 Validation and publication of source files

On 27 September 2026, local validation covered 608 Daily files, 3,664,261 rows and EUR 20,357,040 for January 2025-August 2026. It also covered eighteen `no_change` monthly bills through June 2026, with 3,279,613 rows and EUR 18,220,080. July and August 2026 therefore belong to the validated Daily history, not to this validated monthly-bill set. These figures establish internal consistency of synthetic files, not realistic company spending or successful cloud ingestion. Section 3.2 describes the dataset preparation and validation method.

The same generated Parquet files may feed the local POC and, after a controlled copy, the GCS RAW namespace. The local and cloud processing implementations are described separately in sections 5.2 and 5.3. Section 3.2 discusses the source assumptions and representativeness limit; section 7.4 identifies the cloud run evidence still required.

5.2 - Local End-to-End Proof of Concept

The local POC is a frozen DuckDB/Streamlit experiment. It demonstrates a processing and reporting chain for synthetic January 2025 data without implying that the later Databricks implementation runs the same physical code. Its results are treated as local evidence throughout this thesis; cloud execution is assessed separately in section 7.4.

5.2.1 DuckDB processing chain and data contract

The POC reads detailed monthly Parquet, processes Bronze, Silver and Gold data, and publishes analytical tables into a local DuckDB warehouse. The contract boundary checks the source before canonical Silver publication. This establishes an end-to-end local route from generated bill to queryable output, but it does not validate the behavior of the cloud contract implementation or its operational recovery controls.

For January 2025, the recorded POC outputs contain 164,145 charge lines and EUR 912,000 in `BilledCost` through Bronze, Silver and the Gold fact. The local integrity check reports no orphan foreign keys. Section 7.1 examines these results and their limits.

5.2.2 Gold model and subject-oriented datamarts

The local model preserves the charge-line grain and provides dimensions for recurring cost questions. DuckDB SQL templates publish fourteen materialized datamarts in the `datamart` schema; eleven feed the local dashboard. Subjects include executive cost and volume, cost-basis comparisons, top resources, quality and batch traceability, resource groups, subscriptions, and ownership attribution.

Materialization keeps repeated analytical expressions outside the interface and permits reconciliation with the local source. It also duplicates data and requires coordinated refresh. The local test establishes publication for one synthetic month; it does not establish a measured latency advantage or cloud table-level reconciliation. The cloud project implements corresponding subjects in its own versioned SQL rather than executing the DuckDB templates unchanged.

5.2.3 Local Streamlit dashboard

The POC's Streamlit application reads DuckDB datamarts without writing to them. Its Knowledge Base explains column names and cost formulas alongside the analytical views. Streamlit AppTest exercised eight subviews for January 2025, and the displayed EUR 912,000 billed cost and 164,145 rows reconcile with the local checks. Section 7.1 presents the validation scope. This does not establish effective cloud permissions, Row-Level Security or the behavior of the separate Databricks application.

5.3 - Databricks Cloud Data Platform

The cloud implementation consumes the shared Parquet history but has its own processing code, SQL products and operational controls. It is a PFE platform prototype, not a deployment into Technip Energies' production systems. The technical design below identifies implemented behavior and separates it from cloud outcomes that still need run-level evidence.

5.3.1 RAW, DEV, PROD and OPS organization

The external `finops_raw.landing` volume exposes the same GCS source objects to DEV and PROD. Each business catalog, `finops_dev` and `finops_prod`, has separate Bronze, Silver, Gold and datamart schemas. `finops_ops.audit` keeps operational runs, month status, snapshots, reconciliations and archive events with an explicit `environment` field. Every OPS read and write must preserve that environment distinction.

Bronze retains source values with ingestion metadata, including source-file lineage and run identification. Silver is the canonical validated cost-and-usage representation. Gold provides the charge-line fact and dimensions; fourteen datamarts serve recurring analytical subjects. The central Silver serving table is a copy of the canonical representation, not a second authority. The cloud artifacts are organized under `platform/common`, `platform/serverless` and `platform/classic_compute`; chapter 8 examines what the compute variants mean for cost rather than treating either layout as proof of a cost advantage.

5.3.2 Daily ingestion, backfill and monthly close

The Daily pipeline accepts a source URI and rejects an empty Parquet. It writes Bronze, applies the contract, prepares Silver, requires one open billing month, and refreshes that month in Gold and the datamarts. Source-file lineage prevents a previously ingested path from adding duplicate Bronze or Silver rows. The month remains `OPEN`, and the run identifier links data writes to OPS records. A changed object at an already processed path is not treated as a valid correction; RAW immutability is an operating assumption.

The current Classic Daily Job discovers the oldest RAW Daily file absent from PROD, then processes and validates the same URI in DEV before a promotion gate can admit PROD. Choosing against PROD allows a run interrupted after DEV success to rediscover the file; DEV should then write no additional rows. The template queues runs and limits concurrency to one. It handles one candidate per run, so a backlog requires repeated runs or a sequential catch-up controller. The working completion note reports a first Daily DEV load for 1 July 2026 with PROD promotion disabled; it does not document a full promotion and replay. Section 7.4 records this evidence boundary.

For a closed month, the detailed `billing-YYYY-MM.parquet` file becomes the authoritative source. The close function lands and validates it, checks that it contains the requested month, and records `BEFORE`, `SOURCE` and `AFTER` snapshots. These include row count, monetary sums, account and resource counts, critical-null count, charge-period bounds and, when available, a Delta version. `SOURCE` minus `BEFORE` measures the change from provisional Daily data; it is not automatically an error or saving. The function replaces that month in canonical and serving Silver, verifies `AFTER` row count and `BilledCost` against `SOURCE` within tolerance, requires zero critical nulls, refreshes Gold and datamarts, and sets `CLOSED_DATA_LOADED` under the current non-archiving configuration.

Historical backfill invokes the same monthly-close function sequentially over an inclusive month range. Month-scoped Delta replacement is atomic for each target table operation, but the full sequence across Silver, Gold, datamarts and OPS is not one multi-table transaction. Audit states, reconciliation and rerun procedures are therefore needed after a failure between steps. A dedicated Classic monthly-close DEV-to-PROD Job remains to be constructed and tested. Automatic RAW archival is disabled so that a DEV close cannot remove a bill still needed by PROD; optional archive code is not part of the documented normal close.

5.3.3 Data contract, Gold model and datamarts

Section 4.5 sets out the analytical grain, table inventory and intended business questions. This section identifies the cloud implementation choices and their current evidence boundary.

The versioned FOCUS contract is the gate between Bronze and Silver. It defines expected types, required fields, nullability and selected quality rules. Extra source fields are permitted,

while a missing required field or incompatible type is rejected. Project-specific rules require EUR billing currency and Microsoft as provider, ordered billing and charge period boundaries, and non-null `BilledCost`. Negative billed cost remains valid for credits and adjustments. Pricing and consumed quantities and units are present but optional. These restrictions describe the Azure-derived synthetic corpus, not FOCUS as a universal provider rule.

One remediation is deliberately narrow: an `Adjustment` with null `ServiceName` may use non-empty `ChargeDescription`; unrelated null service names remain invalid. Contract failure prevents Silver publication for that input. Local code and tests document the rule; its exact Databricks execution outcomes still require the run records described in section 7.4.

Gold centers on `fact_finops_cost_usage` at billing-line grain, with ten dimensions for date, billing scope, resource, service, SKU, location, pricing, commitment discounts, charge type and tags, plus a resource/tag bridge. Billing-scope and resource attributes currently receive Type 1 updates; validity columns do not yet prove complete SCD2 history. Versioned cloud SQL defines and loads these structures, while Python passes validated environment identifiers and orchestrates execution. Monthly billing replaces, rather than adds to, the Daily-fed month.

The cloud project defines fourteen subject-oriented datamarts in versioned SQL under `platform/common/sql/datamarts/table_refresh`. Some read Gold; others use central Silver fields not present in the fact. Full refresh simplifies publication but repeats work after each source change. The local POC validated fourteen DuckDB datamarts; that result cannot substitute for table-level DEV/PROD output and reconciliation evidence. The Belgium completion plan reports a successful historical backfill for January 2025-June 2026, but the executed commit, Job and Run IDs, control outputs, row and cost totals, and OPS snapshots must be indexed before the thesis treats it as an independently validated cloud result.

5.3.4 Jobs, audit, validation and recovery

Jobs coordinate environment checks, source selection, loading, promotion and validation. OPS records run status and monthly snapshots independently of the business catalogs. The snapshot sequence distinguishes a source correction from a failed publication: reconciliation requires AFTER to match validated SOURCE, while an interrupted run may leave some downstream tables older than Silver. Recovery therefore starts by locating the run, environment, month, source URI, Delta versions and failed task, then reruns the scoped operation and its controls. A green Job status alone is insufficient evidence of cross-table consistency.

The working completion note reports DEV and PROD backfill controls, but primary run artefacts remain to be assembled. A complete Daily promotion, an idempotent replay, a Classic monthly-close Job and an interrupted-run recovery test need separate records. Section 7.4 defines the validation protocol and current evidence boundary.

5.3.5 Cloud Run Streamlit application and read-only data access

The separate cloud application under `FinOps Cloud Data Platform/apps/finops_dashboard` is packaged with a Dockerfile for Google Cloud Run. It is designed to query `finops_prod.datamart` and selected `finops_ops.audit` objects through a SQL Warehouse without writing to Unity Catalog. Its pages cover the Knowledge Base, executive overview, cost drivers, savings, allocation and accountability, resources, operations and quality, and architecture. The Knowledge Base explains column names and cost calculations; comparisons of `ListCost`, `ContractedCost`, `EffectiveCost` and `BilledCost` remain descriptive until business eligibility and exclusion rules are validated.

The application uses a technical identity for the backend Databricks connection. Its final least-privilege target is `CAN USE` on the SQL Warehouse and `SELECT` on the serving tables and security metadata that the application actually consumes, with no access to RAW, DEV or data-engineering write operations. Credential material must be injected at deployment and not committed to the container image or Git repository. Source code, a Dockerfile and prescribed grants do not establish effective access: the deployed revision, Cloud Run configuration, secret handling, Warehouse permissions and page behavior require retained tests and screenshots. The local Streamlit AppTest result in section 5.2 is not evidence for this cloud deployment.

5.3.6 Proposed role- and scope-based dashboard authorization

The next controlled extension introduces a `security` schema in the environment-independent `finops_ops` catalog. This avoids mixing mutable access rules with the append-oriented operational records in `finops_ops.audit` and avoids duplicating the same dashboard identities in DEV and PROD. `user_entitlement` stores one or more active role-and-scope assignments for a principal. `business_scope` maps the organizational hierarchy to application codes, cost centers, subscriptions or other keys present in the analytical products. The first implementation requires only these two tables; a separate role table is unnecessary while the five role definitions remain versioned in application code.

Role	Scope evaluated by the prototype	Expected visibility
FinOps administrator	Global	All dashboard products allowed to the application
Domain manager	Assigned domain	Applications and projects mapped to that domain
Subdomain manager	Assigned subdomain	Applications and projects mapped to that subdomain
Application owner	Assigned application	Costs and resources of that application

Role	Scope evaluated by the prototype	Expected visibility
Project manager	Assigned project	Costs and resources mapped to that project
Unentitled identity	None	Access denied before analytical queries run

The production target uses Google Identity-Aware Proxy in front of Cloud Run. The application validates the signed IAP assertion, extracts the verified email and uses parameterized SQL to resolve the corresponding entitlement. Because Databricks receives the backend service-principal identity rather than the end-user identity in this design, every analytical query must be constrained by the resolved scope. Page hiding is only a presentation consequence and is not treated as the security control.

For the PFE demonstration, `FINOPS_AUTH_MODE=demo` exposes a selector containing only predefined synthetic personas. It does not provide a free-text email input. `FINOPS_AUTH_MODE=iap` removes the selector and rejects a request whose IAP identity is missing or invalid. The demonstration mode tests authorization behavior; it does not prove SSO, enterprise directory integration or production security. These controls and their tests remain planned until the corresponding SQL, Python code and Cloud Run configuration are implemented.

Source: Google Cloud, Configure IAP for Cloud Run and Getting the user's identity (accessed 1 October 2026).

References

Google Cloud. (n.d.-a). *Configure IAP for Cloud Run*. <https://cloud.google.com/run/docs/securing/identity-aware-proxy-cloud-run> (accessed 1 October 2026).

Google Cloud. (n.d.-b). *Getting the user's identity*. <https://cloud.google.com/iap/docs/identity-howto> (accessed 1 October 2026).